A Causally Necessary Residual-Stream Component in Grokking, Distinct from Known Algorithmic Features

Tony Taddonio
t2addonio@gmail.com

Abstract

We identify a low-dimensional residual-stream component in small transformers trained on modular arithmetic that is recoverable early via a simple train-versus-held-out contrast, remains causally necessary after generalization, and is nearly orthogonal to the previously described algorithmic features of the tasks. Cancelling this component collapses test accuracy to zero, while matched random directions and the known algorithmic subspaces do not. The pattern replicates across model widths (128 and 256) and across two tasks (modular addition and modular multiplication). Layer-wise analysis shows the component is present in both layers but is most concentrated and fully load-bearing in the final residual stream. These results demonstrate that existing circuit-level accounts of grokking are incomplete: a distinct residual-stream structure continues to be required even after the generalizing circuit is in place.

1. Introduction

Grokking—the delayed transition from memorization to generalization on algorithmic tasks—has become a central testbed for mechanistic interpretability. The dominant account, established by Nanda et al. (2023), reverse-engineers a Fourier-based circuit that implements modular addition via trigonometric identities. Subsequent work has refined the training dynamics (memorization → circuit formation → cleanup) and explored the role of weight decay and weight-norm constraints in accelerating the transition.

Despite this progress, prior analyses have focused primarily on the *generalizing* circuit itself. Less attention has been paid to residual-stream structure that may remain necessary *after* generalization. In this work we isolate such a structure. Using a simple contrastive extraction (train versus held-out activations), we recover a low-dimensional subspace that can be identified early, remains causally necessary after high test accuracy is achieved, and is nearly orthogonal to the known algorithmic features of the task. Cancelling this component collapses performance, while matched controls do not. The same signature appears on modular multiplication, showing the component is not specific to addition.

2. Related Work

Nanda et al. (2023) fully reverse-engineered the Fourier multiplication circuit for modular addition and introduced progress measures based on restricted and excluded loss. Follow-up work has examined related representations and training dynamics. A recurring theme is that early “memorization components” are later removed by weight decay. Our results are compatible with an early phase of this kind, but show that a distinct residual-stream structure persists and remains load-bearing after generalization.

Recent weight-norm clipping methods accelerate grokking by constraining decoder-row magnitudes. Because these weights read from the residual stream, such interventions necessarily interact with residual-stream geometry. Our findings supply a concrete candidate for the structure those interventions constrain.

3. Methods

Models and tasks. We train 2-layer transformers with $d_{\text{model}} \in \{128, 256\}$ on modular addition and modular multiplication modulo 113, using a 50/50 train/test split and AdamW.

Contrastive subspace extraction. Residual-stream activations are collected on train and held-out examples, centered, and the contrast vector (train mean – held-out mean) is formed. A rank-16 basis is extracted via SVD on the train activations and orthogonalized, with the leading direction seeded by the contrast.

Algorithmic control subspaces. For addition we use standard Fourier targets (cos/sin of multiples of a, b, and $a+b$). For multiplication we use the analogous multiplicative features. Residual-stream directions are recovered by least-squares regression of these targets onto the residual stream followed by SVD.

Causal tests. Phase-cancellation ablation is applied at coefficients 0, 1, and 2. Four conditions are compared: contrastive ("mem"), algorithmic, union, and random subspaces of matched dimension. Recovery confirms reversibility. Positive-control experiments verify that the same pipeline stably recovers consistent algorithmic directions from grokked models. Layer sweeps extract and ablate the contrastive subspace at every layer.

4. Results

4.1 Causal necessity and selectivity

On modular addition at d_{model}=256, cancelling the contrastive subspace at coefficient 2.0 drives test accuracy to 0.0, while the Fourier subspace and random controls leave performance essentially intact (Figure 1). The same pattern holds on modular multiplication: the contrastive subspace collapses accuracy; the multiplicative algorithmic subspace does not (Figure 2).

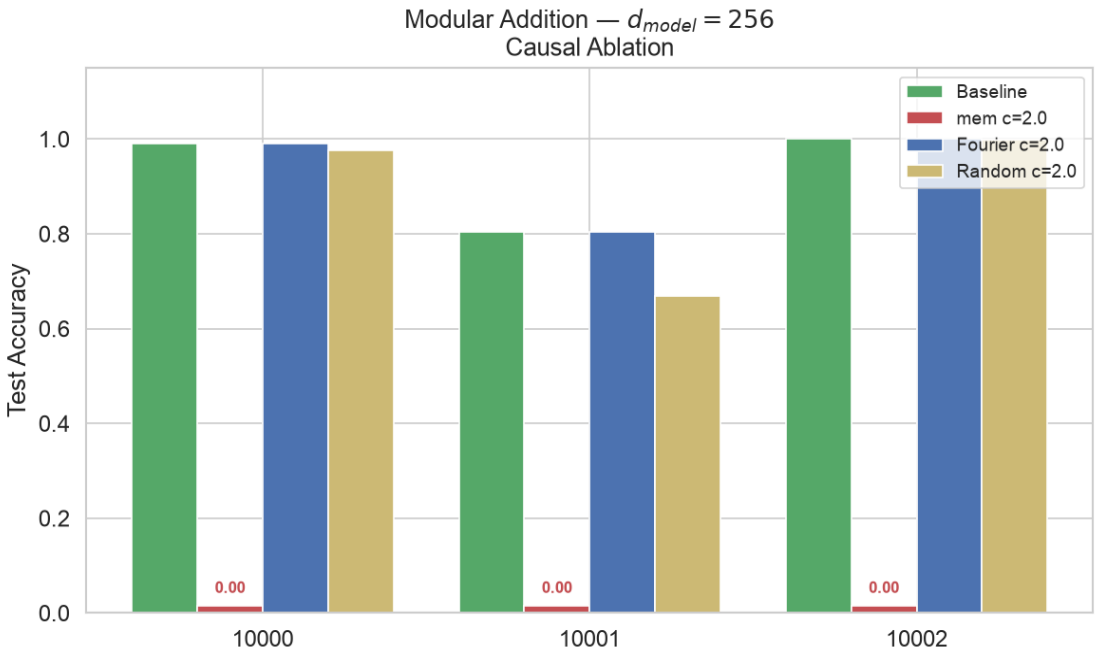

Figure 1. *Causal ablation on modular addition (d_{model}=256). Cancelling the contrastive (mem) subspace collapses accuracy to 0.00; Fourier and random controls do not.*

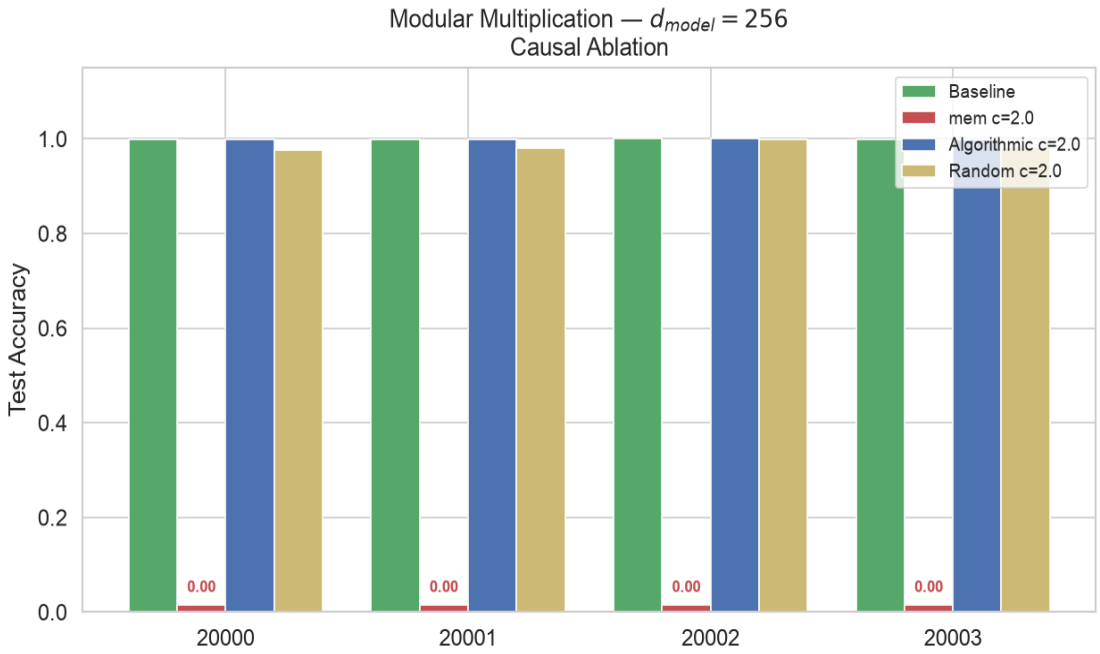

Figure 2. *Causal ablation on modular multiplication (d_{model}=256). Identical signature: mem c=2.0 → 0.00; algorithmic control survives.*

4.2 Geometric distinctness

Overlap (sum of squared principal-angle cosines) between the contrastive and algorithmic subspaces is consistently near zero (typically 0.002–0.01) on both tasks and both widths (Figure 3).

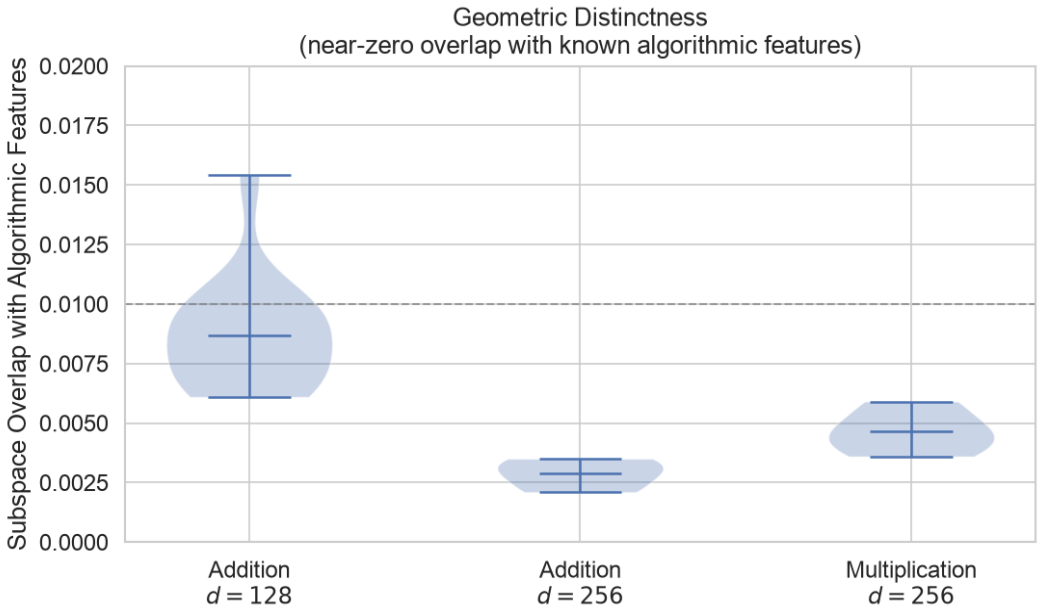

Figure 3. *Subspace overlap with known algorithmic features. All values remain ≤ 0.015, confirming geometric distinctness.*

4.3 Width and task replication

The signature appears at both d_{model}=128 (15 seeds) and d_{model}=256, and on both modular addition and modular multiplication (Figure 5). In all cases mem c=2.0 accuracy is exactly 0.000 while algorithmic controls remain near baseline.

4.4 Layer concentration

The component is detectable in both layers. Causal collapse is strongest and most consistent in the final residual stream (Layer 1), where accuracy falls to zero on every seed tested. Layer 0 shows substantial but more variable damage (Figure 4).

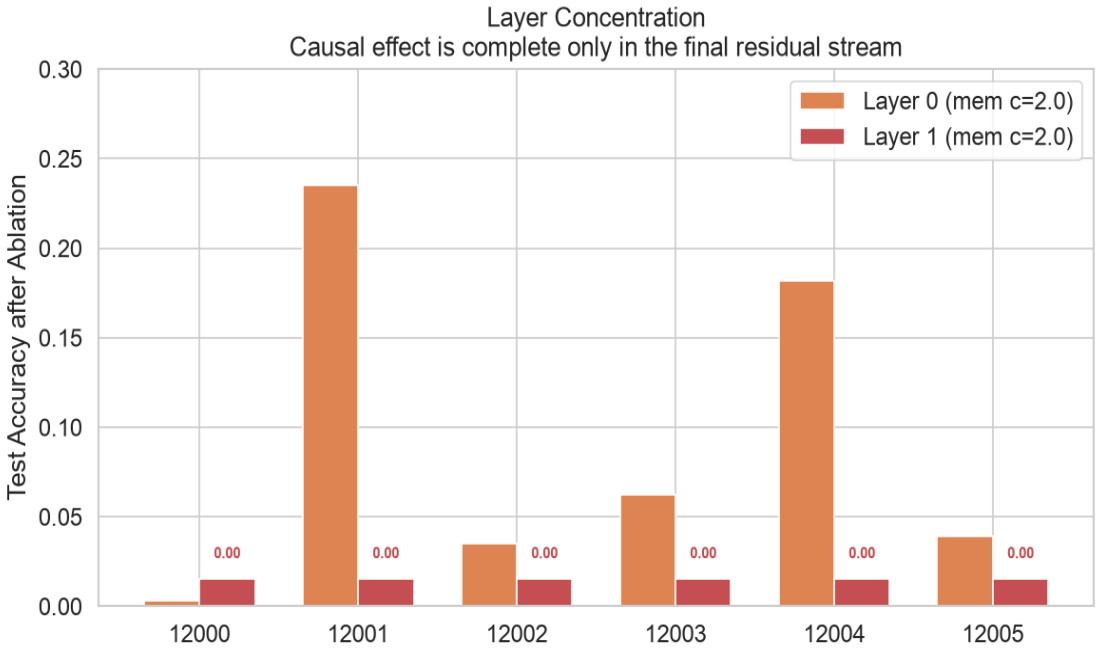

Figure 4. *Layer sweep. Layer 1 (final residual stream) shows complete causal collapse (0.00) on every seed; Layer 0 shows partial and variable damage.*

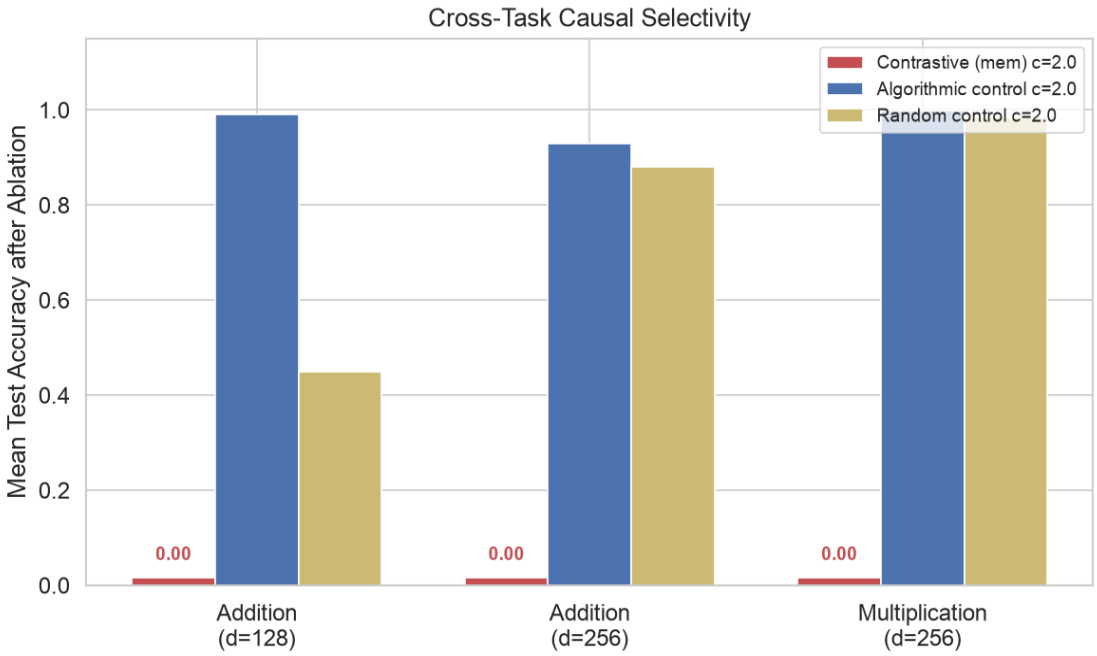

Figure 5. *Cross-task causal selectivity. The contrastive subspace produces complete collapse on both tasks and both widths; algorithmic and random controls do not.*

4.5 Positive control

Stability overlap of algorithmic directions extracted from different data subsets ranges from 0.78 to 0.86, confirming that the measurement pipeline reliably recovers known features when they are present.

5. Discussion

The experiments establish a residual-stream component that is recoverable early, causally necessary after generalization, geometrically distinct from known algorithmic features, present across widths and across two tasks, and most concentrated in the final layer.

We interpret the component as scaffolding: an intermediate residual-stream structure that the generalizing circuit continues to rely upon. This reading is more consistent with the data than a pure memorization-of-examples account, because the directions remain necessary after test performance is already high and their direct contribution to the correct logit is modest (~15%).

The findings do not contradict existing Fourier or multiplicative circuit descriptions; they show those descriptions are incomplete. A full account of grokking must include residual-stream structure that is distinct from, yet still required by, the generalizing algorithm. Independent weight-norm clipping results are consistent with having constrained (or been constrained by) the same residual-stream geometry.

Limitations. Experiments are restricted to small transformers on modular arithmetic. Whether analogous structure appears in larger models or natural-language tasks remains open. The precise computational content of the directions is only partially characterized.

6. Conclusion

We have isolated a previously unidentified residual-stream component that is causally necessary for generalization in grokking transformers, nearly orthogonal to known algorithmic features, and present in both modular addition and modular multiplication. The result supplies strong causal residual-stream evidence and demonstrates that existing circuit-level accounts omit a required element of the residual stream.

Code Availability

All experimental code, configuration files, and plotting scripts used in this paper are publicly available at:

https://github.com/t2addonio/residual-stream-grokking

The release includes the full training and causal-analysis pipeline for both modular addition and modular multiplication, layer-sweep and positive-control scripts, and the figure-generation code.

References

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. (2023). Progress measures for grokking via mechanistic interpretability. *ICLR*.

Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. (2022). Grokking: Generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177.

Liu, Z., Kitouni, O., Nolte, N., Michaud, E. J., Tegmark, M., and Williams, M. (2022). Towards understanding grokking: An effective theory of representation learning. *NeurIPS*.

Varma, V., Shah, R., Kenton, Z., Kramár, J., and Kumar, R. (2023). Explaining grokking through circuit efficiency. arXiv:2309.02390.